

JAVA INTERVIEW PREPARATION

CHAPTER 35

SPRING AOP AND PROXY MECHANICS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Spring AOP and Proxy Mechanics: Interception Boundaries, Self-Invocation, and Advice Ordering

Senior / Lead Java Developer Reading Chapter

Spring AOP applies cross-cutting behavior by exposing a proxy that intercepts eligible method calls and delegates to a target. Transactions, method security, caching, asynchronous execution, retry libraries, and custom aspects often depend on this model. Most surprising failures occur when code calls the target directly rather than crossing the proxy boundary.

Proxy and Target

```
caller -> Spring proxy -> interceptor chain -> target method
```

The bean reference received from the container may be a proxy. Interceptors can run before, after, around, or after failure, then invoke the target.

```
@Aspect
@Component
class TimingAspect {
    @Around("@annotation(Measured)")
    Object time(ProceedingJoinPoint point) throws Throwable {
        long start = System.nanoTime();
        try {
            return point.proceed();
        } finally {
            metrics.record(point.getSignature(), System.nanoTime() - start);
        }
    }
}
```

Around advice must call `proceed` exactly according to intended semantics. Forgetting it suppresses the business method; calling it twice repeats side effects.

JDK and Class-Based Proxies

JDK dynamic proxies implement interfaces and proxy calls through those interfaces. Class-based proxies subclass the target class and can proxy suitable overridable methods.

```
JDK proxy:    proxy implements Service interface -> target
class proxy:  proxy extends Service class       -> target behavior
```

Final classes and final methods cannot be overridden by subclass-based proxying. Private methods are not externally interceptable. Exact proxy choice depends on configuration and framework version; design public service contracts rather than depending on proxy implementation details.

Self-Invocation

```
@Service
class BillingService {
    void bill() {
        audit(); // direct call on this
    }

    @Measured
    void audit() { }
}
```

The call `this.audit()` does not pass through the external proxy, so proxy advice on audit does not run.

```
external caller -> proxy -> bill target
                    |
                    +-> this.audit directly, bypassing proxy
```

Solutions include moving the advised method to a separate collaborator, moving the annotation to the externally invoked boundary, or using AspectJ weaving when genuinely required. Injecting the bean into itself or calling `AopContext.currentProxy()` increases coupling and should be exceptional.

Pointcuts

Pointcuts select join points by method execution, annotation, type, package, arguments, or combinations.

```
@Pointcut("execution(public * com.example.orders..*(..))")
void orderOperations() {}
```

Broad pointcuts can intercept configuration classes, health checks, getters, or internal infrastructure. Keep selection narrow and test which beans are advised. Annotation pointcuts create an explicit marker but can mix infrastructure metadata into domain code.

Spring AOP primarily supports method-execution join points on Spring-managed beans. It does not intercept arbitrary field access, constructors, or objects created with `new`. AspectJ offers broader weaving semantics but introduces build or runtime weaving complexity.

Advice Types

- Before advice runs before the method.
- After-returning runs after successful completion.
- After-throwing observes failure.
- After advice behaves like finally.
- Around advice controls invocation and result.

Use the least powerful advice that fits. Around advice is flexible but can alter arguments, result, exceptions, invocation count, and thread behavior.

Ordering Multiple Interceptors

A method can be transactional, secured, cached, retried, and measured. Order changes semantics.

```
security -> retry -> transaction -> target
```

If retry wraps transaction, each attempt can receive a fresh transaction. If transaction wraps retry, failed attempts may share a transaction already marked rollback-only. If caching occurs before authorization, data may bypass user-specific checks.

Use `@Order` or `Ordered` where supported and write integration tests for the actual chain. Do not assume annotation source order determines runtime order.

Exception Semantics

An interceptor should preserve the original exception unless translation is part of its explicit contract. Catching every `Throwable` and returning null hides failures and may prevent transaction rollback.

```
try {
    return point.proceed();
} catch (DomainException exception) {
    metrics.failure(exception);
    throw exception;
}
```

Be careful with checked exceptions and JDK proxy interfaces. Proxy and reflection layers can wrap undeclared checked exceptions. Service interfaces should declare meaningful failure contracts.

AOP and Object Identity

The proxy and target are different objects. `getClass()` may return a generated proxy class. Equality, serialization, reflection, and annotation lookup can behave differently if code assumes the concrete target class.

Use Spring utilities when framework-aware code needs the target class. Business code should rely on contracts rather than proxy class identity. Never persist a generated proxy class name.

Objects Created Outside the Container

```
BillingService service = new BillingService();
service.bill();
```

This instance has no Spring proxy, injected dependencies, or lifecycle. Entity listeners, serializers, and library callbacks sometimes instantiate objects themselves. Integrate them through supported factories or keep them as ordinary objects that do not require container advice.

AOP Versus Filters, Interceptors, and Decorators

Use the boundary matching the concern:

HTTP protocol concern	-> servlet filter or MVC interceptor
method policy across beans	-> Spring AOP
explicit domain behavior	-> decorator or collaborator
database transaction	-> transaction interceptor at service boundary
arbitrary bytecode joinpoint	-> AspectJ when justified

An explicit decorator is easier to debug when the behavior is part of business meaning. AOP suits infrastructure policy that would otherwise repeat across many services.

Debugging Proxies

When advice does not run, ask:

- Was the object obtained from the `ApplicationContext`?
- Is the call external through the proxy or self-invocation?
- Does the pointcut match the implementation method?

- Is the method interceptable under the proxy mechanism?
- Was proxy creation enabled in the correct `ApplicationContext`?
- Did an earlier `BeanPostProcessor` create or replace the bean unexpectedly?
- Is another interceptor swallowing the exception or returning early?

Log the runtime bean class and inspect proxy/advisor information in a focused test. Avoid enabling extremely verbose AOP logs permanently in production.

How the Proxy Is Created

Proxy creation normally happens during bean post-processing. The container constructs and initializes the target, then an auto-proxy creator evaluates eligible advisors and may replace the exposed bean reference with a proxy.

```
bean definition
-> instantiate target
-> dependency injection and initialization
-> auto-proxy creator finds matching advisors
-> expose proxy reference to other beans
```

The target is still the object on which business code executes. The proxy owns the interception boundary. A bean instantiated too early, before the full post-processor chain is registered, may be reported as not eligible for all post-processors and can miss proxying. This often happens when a custom `BeanPostProcessor` eagerly depends on ordinary application beans.

Proxy configuration is scoped to an `ApplicationContext`. In parent-child contexts, an advisor or auto-proxy creator in one context does not automatically post-process beans created by another. When one controller is advised and another apparently identical bean is not, identify the context that owns each definition.

Advisor Matching Has Static and Runtime Parts

An advisor combines advice with a pointcut. Spring can often determine from method and target class whether the advisor could apply, then evaluate any runtime conditions for each invocation.

```
candidate bean class
-> can pointcut match any method?
-> create proxy and interceptor chain
-> invocation method/arguments may refine runtime match
```

Broad dynamic pointcuts add per-call work and make behavior difficult to inventory. Prefer a stable architectural package, explicit annotation, or known interface. Pointcuts that depend on exact implementation names are fragile under refactoring and generated proxies.

When matching annotations, distinguish the invoked interface method, the implementation method, class-level annotations, meta-annotations, and inherited annotations. The observable result can vary with proxy type and pointcut expression. Test the actual public invocation path rather than assuming an annotation placed somewhere in the hierarchy will be found.

Type Exposure and Injection Consequences

A JDK proxy is assignable to its proxied interfaces, not to the concrete target class. Code that injects or casts the implementation type can therefore fail if the application switches from class-based to interface-based proxying.

```
PaymentService contract = context.getBean(PaymentService.class); // stable
DefaultPaymentService concrete =
    context.getBean(DefaultPaymentService.class); // proxy-sensitive
```

Class-based proxies preserve assignability to the non-final target class but still have method-visibility and finality limits. Neither model makes reflection against `proxy.getClass()` equivalent to inspecting the user class. Framework integration code should resolve the ultimate user type through supported utilities where needed.

This is an architectural reason to inject meaningful contracts, not a reason to create an interface for every helper class. Use an interface when it represents an application boundary, substitution point, or public capability.

Interceptor Ordering Changes Transactional Meaning

Suppose a method has retry and transaction advice.

```
retry outside transaction:
retry -> tx attempt 1 -> rollback
      -> tx attempt 2 -> commit

transaction outside retry:
tx -> retry attempt 1 marks transaction rollback-only
    -> retry attempt 2 runs in same physical transaction
    -> final commit may fail or roll back
```

The first shape is commonly desired for transient database retries because each attempt obtains a clean transaction. It is still safe only when non-database side effects are absent, deferred, or idempotent. Retrying a charge or message publication can duplicate business effects.

Security and caching have similar interactions. Authorization should normally protect every path that can return protected data, including cache hits. A cache key must include security- or tenant-relevant dimensions when results vary by caller. Metrics can wrap the whole chain for end-to-end latency or sit nearer the target for business execution latency; name the measurement accordingly.

Do not solve ordering only with numbers scattered across aspects. Document the intended chain and assert it in an integration test. The semantic diagram is part of the design.

AOP and Thread Boundaries

An interceptor runs in the invocation thread until it deliberately transfers work. Thread-bound transaction, security, logging, and request context do not automatically appear in another executor.

```
caller thread -> proxy -> @Async interceptor -> submit task
                                   |
                                   +-> caller returns
worker thread -----> target executes later
```

With asynchronous advice, exception behavior moves to the returned `Future` or configured uncaught-exception handling for `void` methods. A caller's surrounding transaction does not automatically extend into the worker. If an async target also needs a transaction, the transactional interceptor must execute on the worker-side invocation in the intended order.

Combining `@Async`, retry, transaction, cache, and security on one method produces a difficult chain. Split orchestration from the unit of work so thread transfer, transaction boundary, and retry attempt are visible as separate collaborations.

Return Values, Exceptions, and Around Advice

Around advice must honor the method contract. It should return a value assignable to the declared return type, preserve `null` semantics, and rethrow failures unless translation is explicit.

```
@Around("@annotation(Idempotent)")
Object guard(ProceedingJoinPoint p) throws Throwable {
    RequestKey key = currentKey();
    return store.executeOnce(key, () -> {
        try {
            return p.proceed();
        } catch (Throwable failure) {
            throw new InvocationFailure(failure);
        }
    });
}
```

This sketch illustrates a danger: wrapping `Throwable` can change rollback rules, checked-exception declarations, and error classification. Advice that crosses a functional API boundary must unwrap without losing the original cause and must define what happens when the callback executes zero, one, or multiple times.

For `CompletionStage` methods, the target can return successfully while the asynchronous operation later fails. Traditional after-returning advice sees the stage object, not its eventual outcome. Attach a completion callback when the concern is completion latency or failure, taking care not to consume or replace the stage's result. Reactive publishers have a similar subscription-time model and require reactive-aware operators rather than assuming method return equals work completion.

Proxy Equality and Serialization

Avoid making business correctness depend on proxy identity. Two proxies can wrap one target, or one proxy can resolve changing scoped targets. `equals` and `hashCode` may be delegated or handled by proxy infrastructure according to configuration, but using proxied services as domain keys remains brittle.

Generated proxy classes are runtime implementation details. Do not serialize them into durable messages, persist their class names, or expose them as API schema types. Convert to stable DTOs and inspect the user-facing contract.

Performance and Observability

One proxy invocation is normally small compared with database or network work, but chains on extremely hot in-memory methods can matter. Dynamic pointcuts, argument inspection, stack capture, exception creation, and high-cardinality metrics are more expensive than simple delegation.

Profile before removing policy. If AOP overhead is material, narrow the pointcut, reduce advice work, move measurement to a coarser boundary, or use an explicit decorator whose behavior is easier to optimize. Do not bypass security or transactions for microbenchmark improvement.

Expose the effective chain in diagnostic tests or startup reports without logging sensitive arguments. Useful evidence includes proxy type, target class, advisor names and order, matched method, attempt count, and final outcome.

Testing the Interception Contract

Use three layers:

```
unit test target directly      -> business behavior without advice
focused context test through bean -> pointcut, proxy type, ordering, rollback/cache
integration test with resource -> real transaction/retry/security effect
```

Assert behavior rather than only checking `AopUtils.isAopProxy`. For retry, count committed attempts and side effects. For transactions, verify rollback. For cache, prove key separation. For security, test both direct target logic and proxy-enforced denial. A proxy's existence does not prove the correct advisor matched the correct method.

A Strong Interview Explanation

Spring AOP usually creates a proxy during bean post-processing. Calls entering that proxy are matched against advisors and pass through an ordered interceptor chain before reaching the target. JDK proxies expose interfaces; class-based proxies subclass eligible non-final types. Private, final, or otherwise non-overridable methods cannot be advised by subclass proxying, and self-invocation calls `this` directly, bypassing the external proxy.

A senior answer then connects mechanics to semantics: retry order determines whether attempts get new transactions, async advice moves work to another thread, and completion-aware metrics must observe futures or publishers rather than method return alone. Pointcuts should be narrow, exceptions and return contracts preserved, and tests must call the container-exposed bean. Use AOP for infrastructure policy; use explicit collaborators or decorators when behavior is part of the domain model.

Production Perspective

Proxy mistakes create silent semantic failures: missing transactions, absent authorization, duplicate retries, caches that never populate, or asynchronous methods running synchronously. Unit tests on a raw object cannot verify proxy advice; use a focused Spring integration test for the boundary.

Cross-cutting chains should be observable. Record attempt number, transaction outcome, authorization decision, cache hit, and final latency without exposing credentials or high-cardinality user identifiers.

Interview-Ready Summary

Spring AOP normally exposes a proxy around a Spring-managed target. Only calls crossing that proxy are intercepted. Self-invocation, private methods, final methods under class proxying, and objects created with `new` can bypass advice.

JDK proxies use interfaces; class-based proxies subclass interceptable classes. Pointcuts should be narrow, advice should preserve semantics, and interceptor ordering must be intentional because transactions, retry, caching, security, and metrics interact. Use filters or explicit collaborators when those boundaries better match the concern.

Revision Notes

- Spring AOP commonly uses a proxy plus interceptor chain.
- The container reference may not be the raw target object.
- Around advice controls whether and how often the target executes.
- JDK proxies implement interfaces.
- Class-based proxies subclass eligible target classes.
- Final, private, and otherwise non-interceptable methods limit proxy advice.
- Self-invocation bypasses the external proxy.
- Extracting an advised collaborator is usually cleaner than self-proxy lookup.
- Spring AOP focuses on method execution of managed beans.
- AspectJ supports broader weaving at greater complexity.
- Use the least powerful advice type that fits.
- Multiple interceptors require explicit semantic ordering.
- Annotation declaration order does not define advice order.
- Preserve original exceptions unless translation is intentional.
- Proxy class identity differs from target class identity.
- Objects created with `new` do not receive container proxy behavior.
- HTTP filters and MVC interceptors serve different boundaries from AOP.
- Raw-object unit tests cannot prove proxy behavior.
- Diagnose absent advice by checking object origin, call path, pointcut, and proxy type.
- Auto-proxy creators commonly wrap eligible beans during post-processing.
- Proxy infrastructure and advisors are scoped to their ApplicationContext.
- Annotation matching can differ between interface and implementation methods.
- JDK proxies are not assignable to the concrete target class.
- Retry outside transaction can give each attempt a fresh transaction.
- Async advice transfers execution and usually loses caller thread-bound context.
- Method return is not asynchronous completion for futures or reactive publishers.
- Never persist generated proxy class names or use them as domain identity.
- Test the effective behavior and advisor order, not merely proxy existence.